Engineering Blog

AI

Turbocharging LinkedIn's Recommendation Systems with SGLang

Steven Shimizu
December 9, 2025

Co-authors: Steven Shimizu, Qing Lan, Tejas Dharamsi, Sundara Raman Ramachandran, Arup De, Yubo Wang, Akhilesh Gupta, Yanning Chen, Ata Fatahi, Zhipeng Wang, PhD, and Biao H.

LinkedIn's recommendation engine plays a crucial part in powering key features that shape the member experience. It powers LinkedIn's personalization, helping members see relevant and timely content based on their interests, profession, and activity.

That's why improving recommendation system performance with optimal infrastructure is so crucial. One common pattern in recommendation tasks – such as classification, ranking, and retrieval – is a very long input context with few output tokens (often just one). Those workloads usually come with very high traffic and very low latency requirements. Since SGLang has strong time-to-first-token (TTFT) performance and best-in-class prefix caching, it works great for prefill-heavy workloads. This makes it ideal for use in our reinforcement learning (RL) workflows and LLM for ranking tasks. SGLang is an open-source LLM serving framework with a design that makes it easy to experiment, whether with custom kernels, serving-level optimizations, or alternative model architectures.

In this blog, we'll share how we integrated SGLang into our platform, as well as how this flexibility has fostered an active internal SGLang community at LinkedIn and enabled us to contribute improvements back to the community.

In our LLM-for-ranking design, each request contains a member prompt (system prompt + profile + interaction history) and one or more candidate items (e.g. posts). The model returns the logprobs for a small set of scoring labels (e.g. "Yes" or "No"), which it has been fine-tuned on. These are converted into item scores for ranking. Similar setup and terminologies has been applied in many LLM reranker models like **QWen3 Reranker.**

Multi-item-scoring: Our innovation with SGLang

Figure 1. Comparison of single-item and multi-item scoring approaches.

One of the challenges with SGLang was that using original SGLang required scoring a member's candidates as separate prompts. Even with prefix caching enabled, repeating the same long member context across N member-item prompts adds a huge overhead. This method is termed single-item scoring, represented in the first diagram on the left in Figure 1.

Multi-Item-Scoring (MIS), a key optimization to scoring/ranking applications, helped address this. It is designed to reuse the common prefix to prefill multiple candidate queries. Multi-item scoring allows us to concatenate multiple items together with the member prompt and submit them as a single prompt, thus acquiring the ranking scores in one shot. This is demonstrated in the diagram on the right in Figure 1.

```
<member prefix (system prompt + profile + history)><DELIM><item 1><DELIM><item
2>...<DELIM><item N><DELIM>
```

preference of our members. Then we apply different items, those can be jobs, feed posts and other documents to recommend.

FlashInfer changes

While FlashInfer supported specifying a custom mask to achieve this, the native FlashInfer kernel required the full attention scores to be computed with custom mask patching afterwards. This results in even slower performance than feeding items separately.

To solve this problem, we modified the attention kernel for both flash-attention 2 and flash-attention 3 templates and contributed **the kernel to FlashInfer** and modified SGLang internally to leverage these kernels. These attention kernels accept additional parameters required to use the multi-item scoring attention mask shown in Figure 2 below.

While the shared prefix attends to content as usual, each candidate item segment is bounded by a delimiter token. Attention is restricted, so tokens in one item cannot attend to content from other items. Logits are sampled at item boundaries for the scoring labels.

Figure 2. Multi-item scoring attention mask.

In summary, the following improvements were made:

- Implemented efficient multi-item scoring masks for FA2 and FA3

- Implemented skip tiles for FA2 and FA3 multi-item scoring to improve performance beyond causal mask
- Optimized mask by preloading to L1 cache for thread register

SGLang changes

The SGLang server itself is initialized in multi-item scoring mode with two new parameters: the delimiter token ID and the list of scoring-label token IDs (see Figure 1).

At interference time, our modified FlashInfer attention kernel applies the MIS mask over the concatenated sequence and, for each item boundary, takes the logits at the tokens before the delimiter. We then read the log-probs for the specified label tokens, yielding an N x K matrix (N items x K labels) in the inference result. Positional encoding is also adjusted accordingly to make sure each item aligns with its single item scoring position.

(See our PR contributions to SGLang **here**)

Results

This multi-item scoring optimization dramatically decreased latency of a single request to rank 50 items for a member by **69%,** compared to the baseline single-item scoring approach with prefix caching enabled. Building on top of that, we implemented a multi-item scoring mask in the FlashInfer FA3 attention kernel, which shaved off an additional **~11%** latency versus the prior FA2 implementation.

Beyond our own changes, we also benefited from the continuous

improvements) gave us a further **5%** latency reduction, with no code changes on our side.

Given the significant changes on SGLang, we will incrementally contribute our changes and work with the community to upstream them.

Figure 3. Chart showing the average latency per member ranking request.

In the example provided in Figure 3, the ranking request consists of a 12k token prefix with 50 items (150 tokens each) being ranked. In the single-item scoring (SIS) case, 50 prompts, each containing a single item to score, are sent in a single request to the server with prefix caching enabled. In the MIS case, a single prompt containing 50 candidate items with the delimiter separator is sent in the request.

FP8 kernel improvements

At first glance, one would assume that moving from BF16 to FP8 should double performance. In practice, it's more nuanced. FP8 quantization only applies to linear layers (e.g., MLPs and the QKV/output projections in attention).

We experimented with online FP8 quantization, where activations are quantized on the fly. Instead of a single GEMM kernel, the linear layer becomes three kernel launches:

1. Segmented max reduction (to find the per-tensor scaling factor)

3. GEMM on FP8 inputs

Although the GEMM itself is faster in FP8, the extra preprocessing steps (reduction and scaling) not only offset those gains, but actually led to slower overall latency (+7.1%) than plain BF16.

Accuracy was also a challenge. The early FP8 kernel used by SGLang used only a per-tensor scaling factor, which resulted in noticeable degradation on ranking metrics. While generative use cases may not see a noticeable effect, because the candidate item score is the relative probability of a single token, we are much more sensitive to any accuracy loss. For our use case, we care about NDCG@1 (ensuring the top-ranked item remains consistent with the BF16 baseline).

Figure 4. Illustration showing the different scaling factors for quantization. (Ref: https://lilianweng.github.io/posts/2023-01-10-inference-optimization/)

The breakthrough came with the new sgl_per_token FP8 kernel in SGLang. Scaling at a finer per-token level improved accuracy and delivered real performance gains (**-9.0%**) compared to BF16.

Figure 5. FP8 performance metrics

(See our PR contributions to SGLang here: **1**, **2**, **3**)

FlashAttention 3 integration in SGLang

that was initiated and contributed by the LinkedIn team to the SGLang open-source project.

Why was this important? Many of LinkedIn's RecSys workloads are prefill-heavy, meaning they have extremely long input contexts (often tens of thousands of tokens) but generate very few output tokens. In such cases, the attention kernel is the performance bottleneck, and small efficiency gains translate directly into better latency and throughput.

By building and upstreaming the FA3 backend, we:

- **Enabled all major attention features in one unified backend** (CUDA Graph, speculative decoding, sliding window, FP8 support, multimodal support, etc.) so production teams no longer need to rely on fragmented or model-specific solutions.
- **Delivered strong speedups for long-context inference** thanks to FA3's optimized tiling and parallelization – perfectly suited to recommendation ranking tasks.
- **Standardized performance across models** so LinkedIn can serve diverse workloads on a single high-performance stack.

Now SGLang has made FA3 the default, production-ready attention backend on Hopper GPUs in SGLang OSS. This not only benefits LinkedIn internally but also strengthens the open-source ecosystem for everyone building on top of SGLang.

Engineering Blog

Recommendation systems are usually multi-step processes involving high recall candidate retrieval, ranking, post-processing filters, etc. There are two sets of features for ranking: user features and item features. User features (e.g. member profile, previous interaction history) included in the prompt stay constant regardless of which candidate items are retrieved. Item features (job content, feed content) consist of the content that the system must assess and rank for a given user.

Because of this, we preemptively run the LLM on the user context while retrieval is executing, effectively hiding the large member-prompt prefill behind item retrieval. When candidates arrive, we append the items to the prompt and issue a second request to the same SGLang instance, reusing the prefilled KV cache from the first request. We do this using a streaming gRPC connection from the client to the SGLang instance, where we send the member context in the first chunk and the item context in the second chunk. This second request is much faster than a cold start, reducing overall perceived latency. In our use case, we observed a significant ~38% decrease in overall latency (from 520ms to 200ms).

This "knock-knock" approach does consume more GPU compute (two requests), but it's a favorable trade-off for latency-sensitive applications.

To support this and ensure the follow-up call hits the correct SGLang DP (data-parallel) worker with the right KV cache, we modified SGLang to return the DP rank for each request and to accept a DP rank as input, overriding round-robin routing.

Figure 7 shows two contrasting timelines for recommendation serving:

1. **Without Knock-Knock**: The item retrieval occurs first, which is input into constructing the prompt used for the ranking request.
2. **With Knock-Knock:** The member prefill step is performed in parallel to the item retrieval step (knock 1). The second request containing user and item features leverages the cached member prompt KV and completes in a significantly shorter time.

Conclusion

As of today, many of the above features have been upstreamed to SGLang and FlashInfer. These improvements from the kernel to the systems level helped us to finally achieve 3-4x reduction in TTFT latency and better support better ranking use cases. Those capabilities allowed us to reduce operational overhead to maintain our internal fork of SGLang and also help us to build our in-house stack easily – and we're continuing to make further improvements to the SGLang stack.

For our LinkedIn members, the LLM-based recommendation system allows us to provide more customized experience. Through understanding the preferences and browning history of our members, we can recommend more content and topics that our members care about, ultimately creating a better experience on LinkedIn.

Topics: Artificial intelligence Open Source Scalability

Engineering Blog

AI

AI helping build better AI: How agents accelerate Liger Kernel...

Chen Zhu | Apr 21, 2026

Managing software license lifecycles: the SLLM journey

Karthikeyan Subramanian | Apr 14, 2026

Engineering Blog

Marketing

Building LinkedIn's CTV Ads: Scaling professional reach to the...

Albert Hwang | Apr 3, 2026

© 2026

About

Accessibility

User Agreement

Privacy Policy

Cookie Policy

Copyright Policy

Brand Policy

Guest Controls

Community Guidelines

Language